FACULTAD DE INGENIERIA
Universidad de Buenos Aires

# *Wikipedia Article Analysis*

*5 de noviembre de 2026*

Tomás Pérez D'Angelo (tperezd@fi.uba.ar)
Ignacio Politti (ipolitti@fi.uba.ar)
María Florencia Paisan (mpaisan@fi.uba.ar)

Facultad de Ingeniería, Universidad de Buenos Aires

**Descripción General del Sistema**

El presente documento define la arquitectura para un sistema distribuido diseñado con el propósito de procesar y analizar grandes conjuntos de artículos de Wikipedia en los idiomas inglés y francés. El sistema está estructurado para optimizar la ingesta de datos y resolver de manera eficiente consultas sobre el contenido y metadatos de los documentos.

# Índice

**1. Introducción** **3**
1.1. Consultas . . . 3
1.2. El dataset . . . 4

**2. Objetivos y restricciones arquitectónicas** **4**
2.1. Objetivos arquitectónicos . . . 4
2.2. Restricciones y supuestos . . . 4

**3. Vista lógica** **4**
3.1. DAG . . . 4

**4. Vista de Procesos** **6**
4.1. Diagrama de Actividades . . . 6
4.1.1. Comunicación por batches . . . 6
4.1.2. Coordinación y sincronización de fin de datos . . . 6
4.2. Inicio del flujo . . . 7
4.2.1. Consultas 1 y 2 . . . 8
4.2.2. Consultas 3 y 4 . . . 8
4.2.3. Consultas 5 . . . 10
4.3. Diagrama de Secuencia . . . 10

**5. Vista de Desarrollo** **11**
5.1. Diagrama de Paquetes . . . 11

**6. Vista física** **13**
6.1. Despliegue . . . 13
6.2. Robustez . . . 14

**7. Escenarios** **14**
7.1. Procesamiento de una ejecución . . . 14
7.2. Ejecuciones concurrentes . . . 15
7.3. Terminación ordenada . . . 15

**8. Tamaño y performance** **15**
8.1. Configuración de la prueba . . . 15
8.2. Resultados observados . . . 15

**9. Listado de tareas** **16**
9.1. Tareas por integrante . . . 16

**A. Apéndice** **16**
A.1. Siglas y abreviaturas . . . 16
A.2. Definiciones . . . 17
A.3. Principios de diseño . . . 17

# 1 Introducción

Este informe presenta el análisis y diseño de un sistema distribuido destinado a procesar artículos de Wikipedia en inglés y francés, respondiendo a distintos tipos de consultas.

Para describir la solución propuesta, se presentan distintos diagramas y explicaciones acerca de todos los componentes del sistema y los distintos flujo.

## 1.1 Consultas

El sistema responderá las siguientes cinco consultas sobre el conjunto de artículos en inglés y francés:

1. **Artículos en inglés hasta 2020:** obtener la URL y fecha de modificación para artículos escritos en inglés hasta 2020 inclusive.

2. **Artículos con secciones biográficas:** obtener el nombre, url e imagen (bytes) de artículos escritos hasta 2020 inclusive que incluyan "biography" o "biographie.$^{en}$tre sus secciones.

3. **Vocabulario compartido:** obtener un listado de palabras únicas compartidas entre cualquier artículo escrito en inglés y cualquier artículo escrito en francés.

4. **Palabras más frecuentes en los resúmenes:** devolver los pares (palabra, cantidad) del top 20 palabras más utilizadas entre todos los abstracts. Dentro de un mismo abstract las palabras se cuentan una sola vez y las palabras triviales (stopwords) se excluyen del top.

5. **Artículos recientes con resúmenes extensos:** obtener la URL y cantidad de palabras de abstract para los artículos escritos a partir del 2021 cuya cantidad de palabras en el abstract sea superior al promedio de palabras de abstract de todos los artículos escritos hasta el 2020 inclusive. Se aplica el mismo criterio para contabilizar palabras que la consulta anterior.

### 1.2 El dataset

El cliente interactúa con el sistema mediante el envío de artículos de Wikipedia en los idiomas inglés y francés. Cada artículo se representa con los siguientes campos:

| Campo | Contenido |
|---|---|
| `abstract` | Resumen del artículo. El valor puede no estar presente. |
| `date_modified` | Fecha y hora de la última modificación registrada para el artículo. |
| `image` | Referencia a la metadata de la imagen. El valor puede no estar presente. |
| `in_language` | Idioma del artículo, identificado como `en` (inglés) o `fr` (francés). |
| `name` | Título o nombre del artículo. |
| `sections` | Estructura que describe las secciones del artículo. |
| `url` | Dirección web del artículo en Wikipedia. |

Tabla 1: Campos de cada artículo del dataset

## 2 Objetivos y restricciones arquitectónicas

### 2.1 Objetivos arquitectónicos

- **Escalabilidad:** permitir aumentar la capacidad agregando réplicas.
- **Procesamiento concurrente:** procesar ejecuciones de distintos clientes en paralelo y mantener separados sus datos y resultados.
- **Desacoplamiento:** utilizar un middleware que abstraiga la comunicación mediante colas y mensajes.
- **Cierre ordenado:** responder a `SIGTERM` deteniendo los componentes sin perder los mensajes en viaje.

### 2.2 Restricciones y supuestos

- El sistema procesa artículos en inglés y francés y utiliza los campos disponibles en el dataset.
- Se atienden múltiples clientes, pero no se requiere ejecutar más de una consulta simultánea por cliente.
- La consulta de imágenes depende de la disponibilidad del servicio de Wikimedia. El dataset puede no incluir una imagen para todos los artículos.

## 3 Vista lógica

### 3.1 DAG

El siguiente diagrama muestra el flujo del procesamiento de los datos:

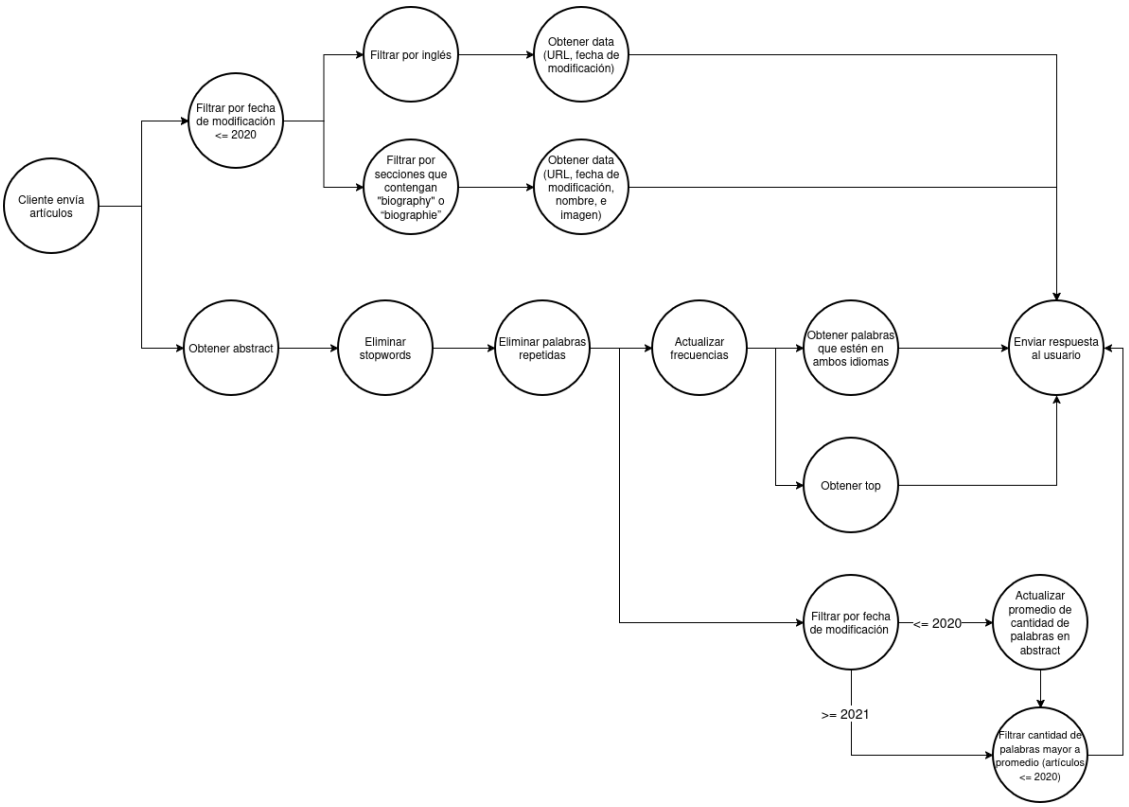



Figura 1: Flujo lógico de procesamiento de artículos

El flujo procesa los artículos enviados por el cliente dividiéndose en dos ramas principales según el tipo de análisis requerido:

- **Rama de metadatos (Consultas 1 y 2):** Aplica un filtro inicial sobre la fecha de modificación ($\leq$ 2020) y bifurca para aplicar un filtro por idioma (inglés) para la consulta 1 y otro por secciones ("biography"/"biographie") para la consulta 2.
- **Rama de procesamiento de abstract (Consultas 3, 4 y 5):** Extrae los *abstracts* y aplica una limpieza (eliminación de *stopwords* y palabras repetidas) para:
  - Actualizar las métricas para el posterior cálculo del promedio de longitud de abstract de los artículos hasta el 2020 inclusive, para filtrar aquellos posteriores a 2021 que superen dicha media
  - Construir la tabla de frecuencias de palabras a partir de la cuál se obtienen las palabras compartidas entre idiomas y el ranking de palabras más frecuentes (*top*)

Finalmente, todos los flujos convergen en el nodo final que envía los resultados al usuario.

# 4 Vista de Procesos

Describiremos en esta sección el comportamiento del sistema durante su ejecución: qué componentes trabajan, qué información intercambian y cómo se coordinan para atender las consultas.

## 4.1 Diagrama de Actividades

Los diagramas de actividades representan el flujo de trabajo mediante acciones y decisiones. Se realizaron distintos diagramas para las distintas etapas por las que pasa el sistema.

### 4.1.1 Comunicación por batches

Se implementó comunicación por *batches* de mensajes entre nodos para optimizar el uso de las conexiones, reduciendo el overhead por mensaje.

En los siguientes diagramas se representa una version simplificada en la que no se reciben *batches*. Esto con el fin de facilitar la visualización de la actividad central.

### 4.1.2 Coordinación y sincronización de fin de datos

Como una etapa puede ejecutarse en varias instancias, recibir `EOF` en una sola de ellas no garantiza que todas hayan terminado de procesar los mensajes de un cliente.

En las etapas donde se divide el trabajo a realizar para un cliente en diferentes instancias, es necesario coordinar el cierre para que el envío del `EOF` sea único por cliente por etapa.

Con este propósito se implementó el módulo *Synchronizer* que mantiene el estado de cada conexión que requiere sincronizar el envío de un único `EOF` entre múltiples instancias.

- **Conteo de registros:** aunque los artículos se transporten en *batches*, el progreso se cuenta por artículo. El cliente incluye en el `EOF` inicial la cantidad total enviada. En las etapas posteriores, el total esperado debe corresponder a los mensajes de entrada de esa etapa.
- **Difusión y coordinación:** al recibir el EOF de una etapa, el *Synchronizer* notifica a las instancias mediante un intercambio de sincronización. Cada instancia informa cuántos artículos procesó, una instancia coordinadora reúne los reportes y verifica que estén representadas todas las instancias.
- **Barrera de cierre en dos fases:**
  1. Cuando los contadores confirman que se procesó el total esperado, cada instancia publica los mensajes de salida que tenga pendientes y vacía sus buffers.
  2. Una vez que todas las instancias confirman que terminaron de publicar sus resultados, el coordinador envía un único `EOF` a la etapa siguiente, indicando la cantidad de mensajes enviados.

Así se evita que una señal de fin llegue a la etapa siguiente antes que los datos producidos por las instancias. `EOF` indica el fin de los datos de una ejecución; `SIGTERM`, en cambio, solicita la terminación ordenada de un proceso.

## 4.2 Inicio del flujo

El primer diagrama presenta el comienzo del procesamiento y separa las responsabilidades del cliente y de *Article Handler*:

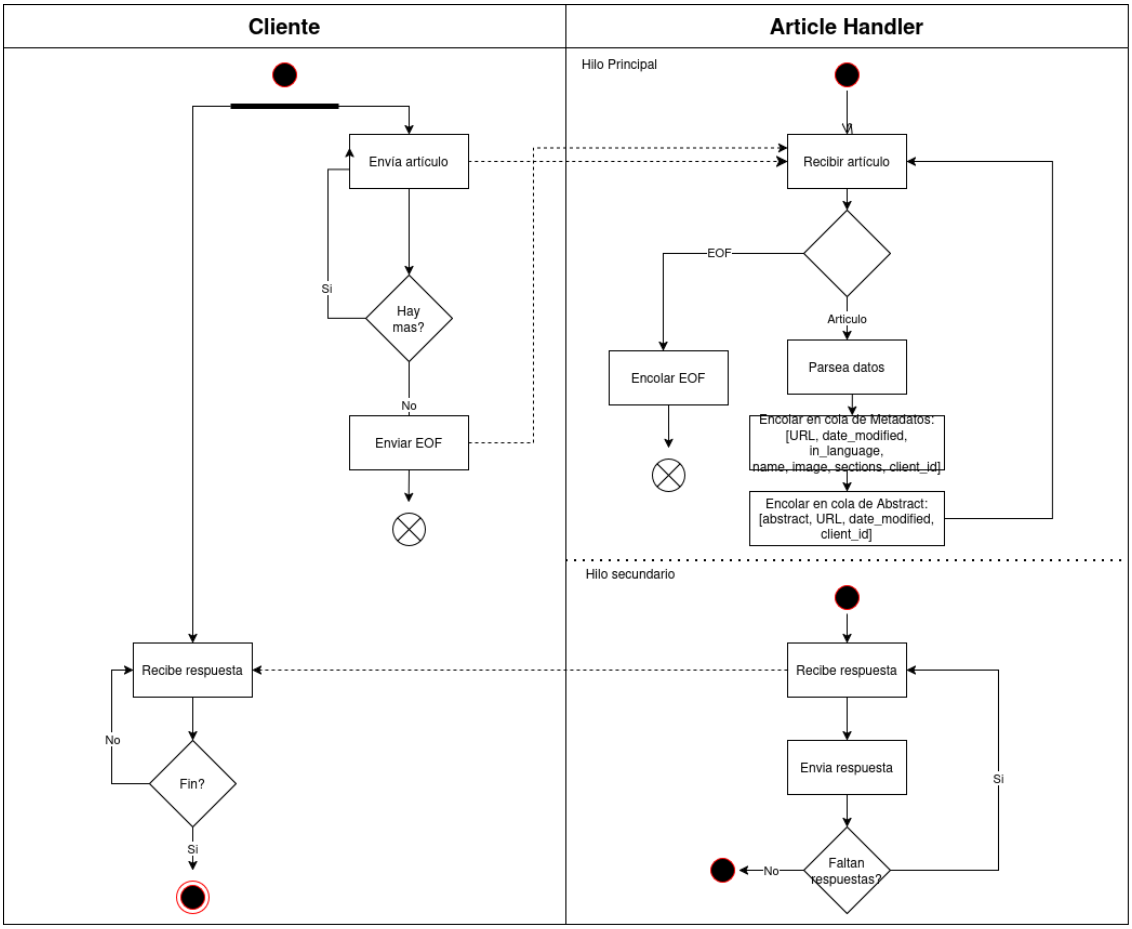



Figura 2: Inicio del flujo de ingreso de artículos

El flujo comienza con el envío de artículos desde el cliente, que los remite a *Article Handler*. Por cada artículo, el handler parsea sus datos y genera dos mensajes: uno con los metadatos (URL, fecha de modificación, idioma, nombre, imagen y secciones) y otro con el abstract (resumen, URL y fecha de modificación). Ambos incluyen el identificador del cliente y se colocan en colas distintas para que las etapas posteriores consuman la información que necesitan. El ciclo se repite hasta que el cliente no tiene más artículos y envía `EOF`; el handler encola este marcador para señalar el fin de la ingesta. En paralelo, un hilo secundario recibe las respuestas generadas y las envía al cliente hasta que no quedan más.

### 4.2.1 Consultas 1 y 2

El diagrama siguiente describe el procesamiento de los metadatos y la obtención de imágenes:

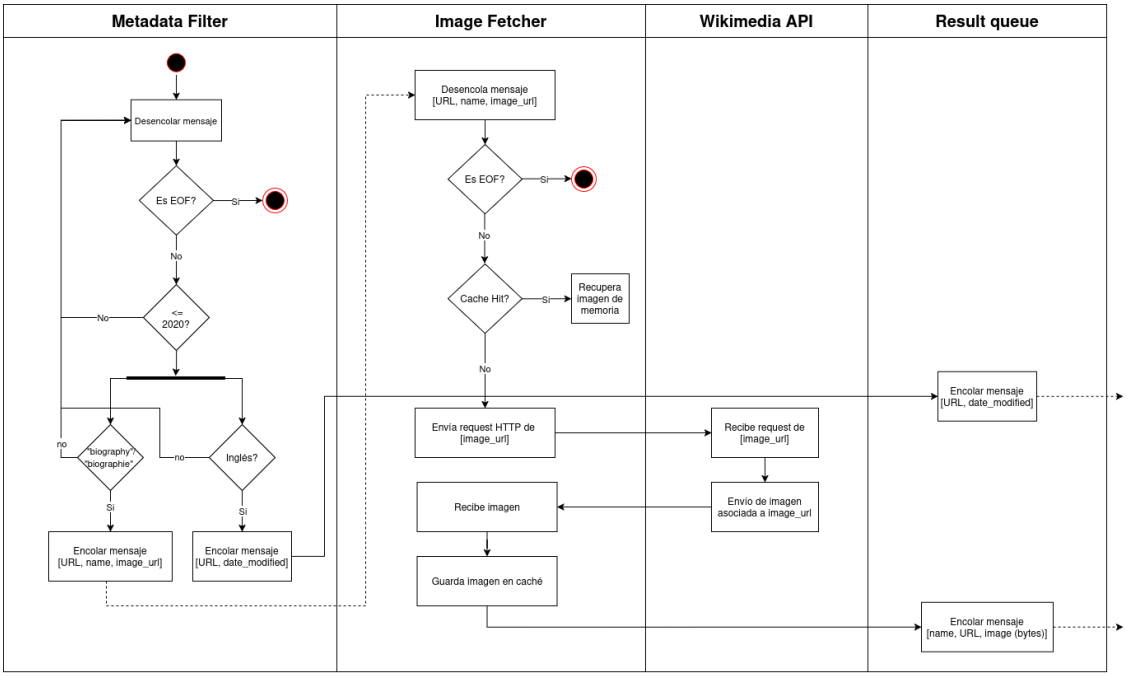



Figura 3: Filtrado de metadatos y obtención de imágenes

*Metadata Filter* consume los mensajes de la cola de metadatos y verifica si recibió `EOF`. Para cada artículo con fecha de modificación válida hasta 2020 inclusive, si el idioma es inglés, encola en la cola de resultados la URL y la fecha de modificación (consulta 1). Luego, si el artículo tiene una sección `biography` o `biographie`, deriva a *Image Fetcher* la URL del artículo, el nombre y la URL de la imagen (consulta 2).

*Image Fetcher* solicita la imagen mediante un GET HTTP a la URL provista por Wikimedia, recibe su contenido y si la descarga fue exitosa, encola un resultado con el nombre, la URL del artículo y los bytes de la imagen. No cualquier instancia recibe cualquier URL: el *Metadata Filter* calcula una partición sobre la URL de la imagen y módulo la cantidad de instancias, de modo que la misma URL siempre llega a la misma instancia de *Image Fetcher*.

**Descarga de imágenes y caché.** La descarga se realiza con un GET HTTP a la URL de la imagen provista por Wikimedia. Cada instancia de *Image Fetcher* tiene un caché LRU en memoria de 100 entradas. Antes de descargar, el nodo consulta su caché: en un *Cache Hit* evita la petición HTTP, y en un *Cache Miss* descarga respetando el User-Agent configurado, un timeout de 30 segundos, hasta 5 intentos con backoff exponencial y el `Retry-After` ante HTTP 429.

### 4.2.2 Consultas 3 y 4

Para estas consultas se procesan los abstracts de los articulos y se trabaja en base a la información extraida.

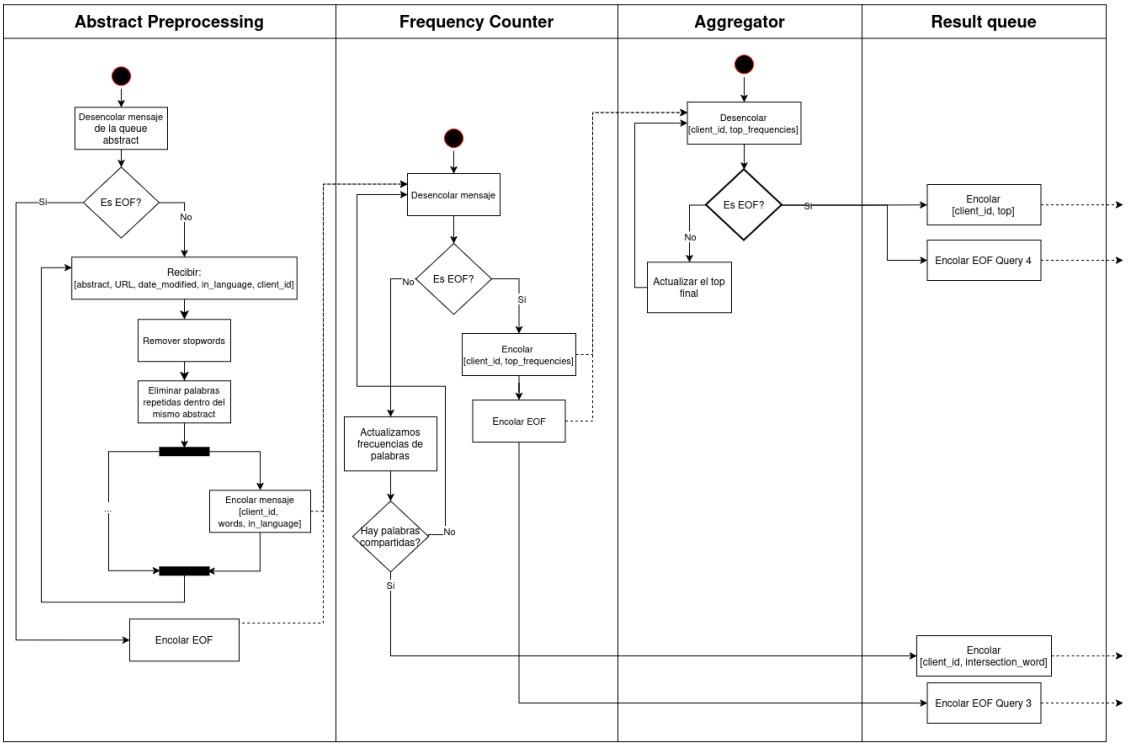



Figura 4: Preprocesamiento de abstracts y cálculo de queries 3 y 4

*Abstract Preprocessing* Recibe los abstracts de los artículos junto con su URL, fecha de modificación e idioma. Para cada abstract normaliza el texto a minúsculas, tokeniza las palabras (letras, números y guion bajo), descarta las *stopwords* y los tokens puramente numéricos, y elimina las repeticiones dentro del mismo abstract. Luego genera mensajes para tres destinos: el exchange de palabras hacia *Frequency Collector* (consultas 3 y 4), el exchange *abstractsUntil2020* hacia *Average Calculator* y la cola *abstractsFrom2021* hacia *Abstract Word Counter* (consulta 5). Para las consultas 3 y 4 nos interesa el primer destino: envía las palabras procesadas y el idioma del artículo. Las palabras se particionan por cliente y palabra entre las instancias de *Frequency Collector*. El `client_id` se incluye en todos los mensajes para conservar la identificación del flujo.

*Frequency Collector* mantiene, por cliente y por palabra, las frecuencias en inglés y francés; ignora los artículos cuyo idioma no sea `en` o `fr`. Al finalizar la ingesta de un cliente (cuando la sincronización confirma que todas las instancias de preprocesamiento terminaron de enviarle), envía su top parcial de palabras por frecuencia a *Top Aggregator*. Este combina los tops parciales en uno final y, cuando recibió el fin de todos los emisores esperados, envía la lista de palabras más frecuentes a la cola de resultados para resolver la consulta 4. Durante la ingesta, el colector también detecta las palabras que aparecen por primera vez en el otro idioma y las envía en ese momento a la cola de resultados de la consulta 3; cada palabra se envía una sola vez. Como el preprocesamiento elimina duplicados por abstract, una palabra se contabiliza como máximo una vez por artículo.

### 4.2.3 Consultas 5

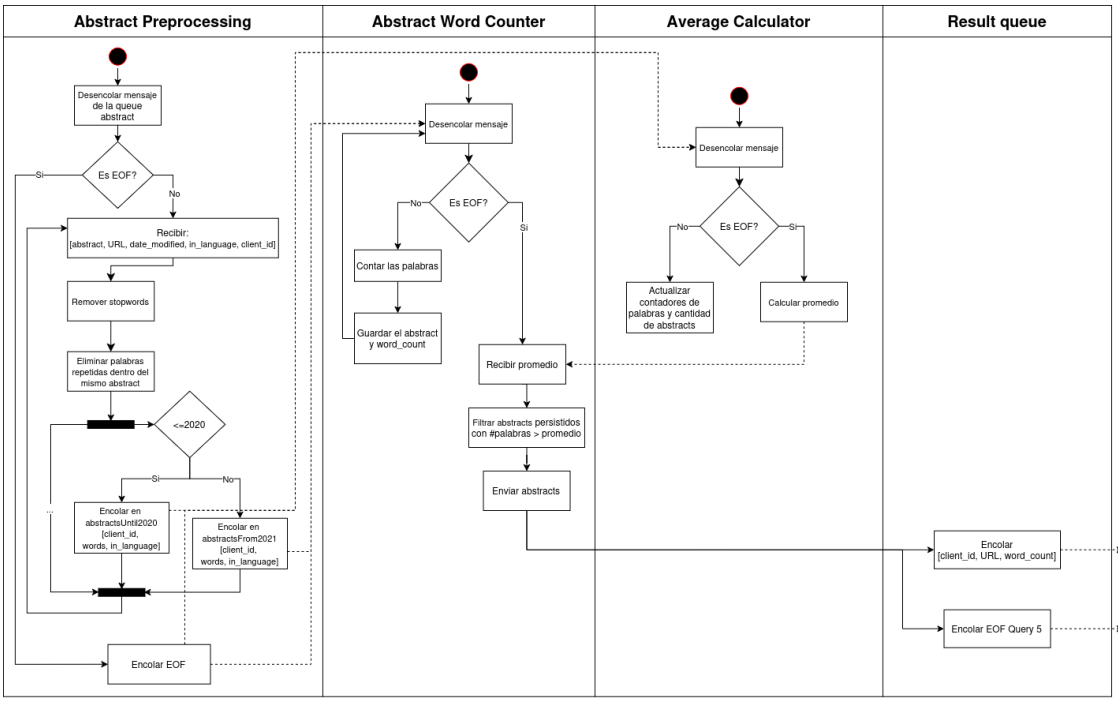



Figura 5: Preprocesamiento de abstracts y cálculo de query 5

*Abstract Preprocessing* Luego de aplicar el preprocesamiento explicado antes, envía a una de las siguientes etapas el recuento de palabras únicas del abstract junto con la URL y el `client_id`: a *Average Calculator* si el artículo tiene fecha de modificación hasta 2020 inclusive, y a *Abstract Word Counter* si es posterior. La decisión se toma según el año de `dateModified`: si la fecha no es válida, el abstract no se envía a la consulta 5.

Por otro lado, *Average Calculator* acumula, por cliente, la cantidad total de palabras y la cantidad de abstracts. Las instancias se reparten los clientes por partición. Cuando recibió el `EOF` y procesó todos los abstracts esperados, calcula el promedio (palabras totales dividido la cantidad de abstracts) y lo envía al exchange *averages*, que consumen todas las instancias de *Abstract Word Counter*.

*Abstract Word Counter* almacena los abstracts posteriores a 2020 y su cantidad de palabras, esperando el promedio calculado para cada cliente. Por cada artículo guarda en disco el par URL–cantidad en un archivo propio del cliente. Cuando ya recibió el promedio y la sincronización le confirma que todas las instancias de *Abstract Word Counter* procesaron los conteos esperados del cliente, recorre su archivo y envía a la cola de resultados solo los abstracts cuya cantidad de palabras es estrictamente mayor que el promedio. Luego elimina el archivo y emite el `EOF` final, resolviendo la consulta 5.

## 4.3 Diagrama de Secuencia

El diagrama de secuencia complementa los diagramas de actividades: muestra quién se comunica con quién y el orden temporal de los intercambios de mensajes.

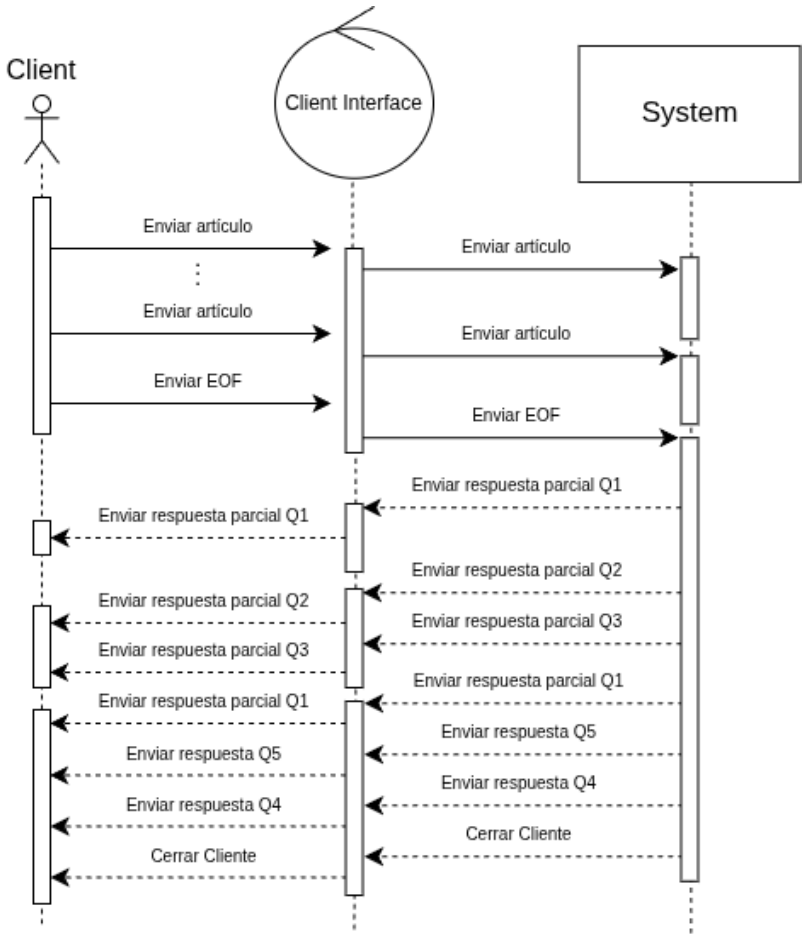



Figura 6: Intercambio de mensajes entre el cliente, la interfaz y el sistema

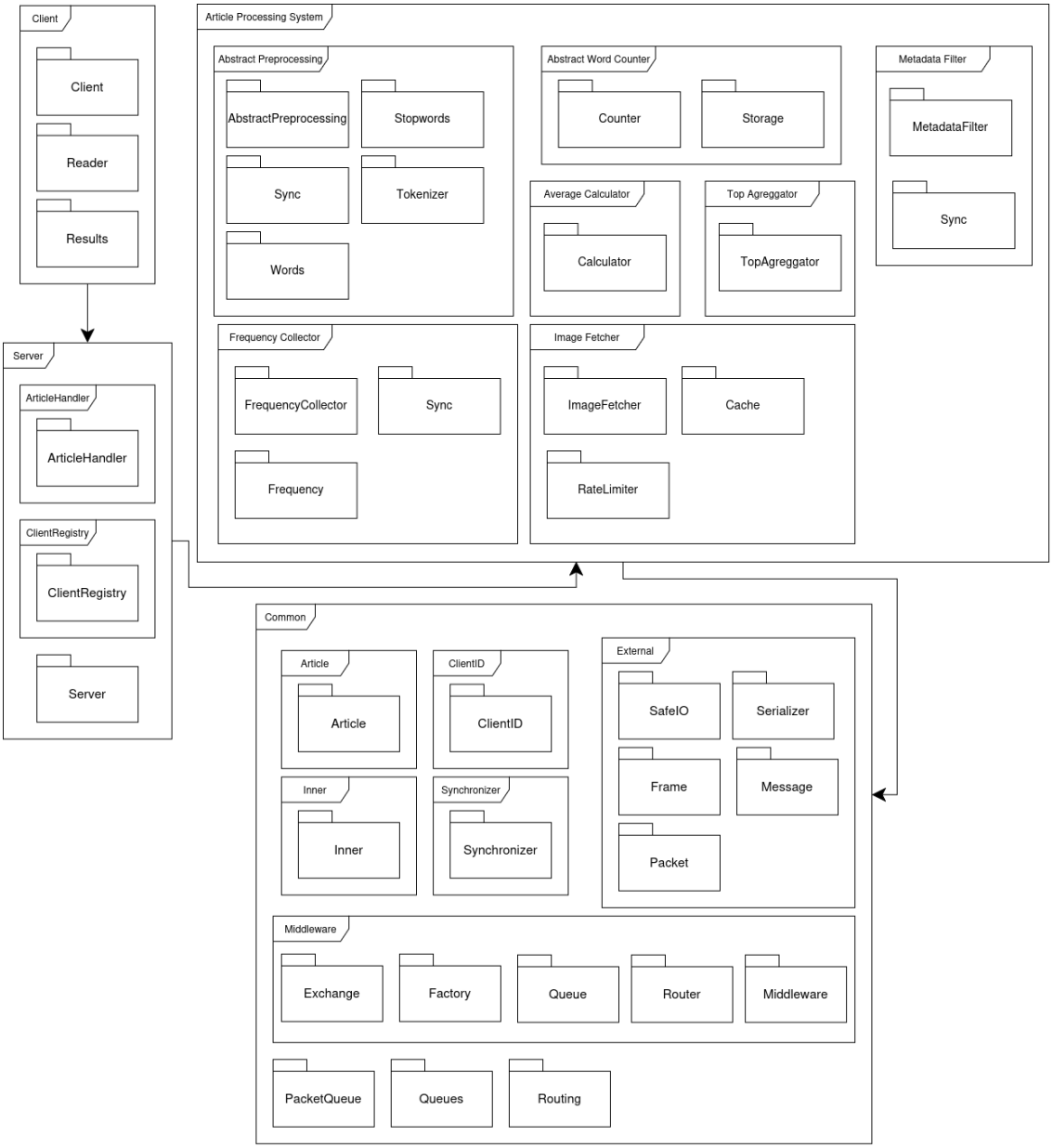
El cliente envía uno o más artículos a *Client Interface*, que los reenvía al sistema para su procesamiento. Al terminar el envío, transmite `EOF`. El sistema devuelve las respuestas a la interfaz, que las entrega al cliente y, al finalizar, recibe el mensaje de cierre.

Las consultas 1, 2 y 3 pueden enviar resultados parciales, por lo que sus componentes deben coordinar el fin del procesamiento mediante mensajes `EOF`. En cambio, las consultas 4 y 5 responden una vez completados los cálculos que requieren procesar todos los artículos.

# 5 Vista de Desarrollo

## 5.1 Diagrama de Paquetes

El siguiente diagrama muestra la organización del sistema en paquetes y sus relaciones. Cada paquete agrupa componentes vinculados con una responsabilidad en común.




El paquete *Client* agrupa componentes de entrada e interacción. *DataIngestor* incorpora los artículos al sistema, mientras que *ClientInterface* sirve como punto de comunicación con el procesamiento de artículos.

*Article Processing System* contiene los módulos que realizan el procesamiento principal de los artículos.

El paquete *ArticleHandler* contiene el componente que recibe y coordina el tratamiento de los artículos.

El paquete *Calculation* agrupa *FrequencyCounter* y *AverageCalculator*. Estos componentes calculan las frecuencias de las palabras y promedios.

El paquete *Aggregation* contiene *TopAggregator*, que agrupa los resultados producidos por el procesamiento y permite organizarlos.

El paquete *Metadata Processing* incluye *MetadataFilter*, encargado de trabajar con los metadatos de los artículos y aplicar los criterios de filtrado requeridos por las consultas.

El paquete *Abstract Processing* contiene *PreprocessingModule*, que prepara los resúmenes para los análisis y cálculos posteriores.

El paquete *Middleware* implementa la comunicación entre componentes mediante *RabbitMQ*, que actúa como intermediario de mensajes.

Por último, el paquete *Wikimedia* encapsula la interacción con la *Wikipedia API* mediante el *ImageFetcher*. Este componente obtiene las imágenes asociadas a los artículos.

# 6 Vista física

La vista física muestra dónde se ejecutan los componentes del sistema y cómo se comunican entre sí y con servicios externos.

## 6.1 Despliegue

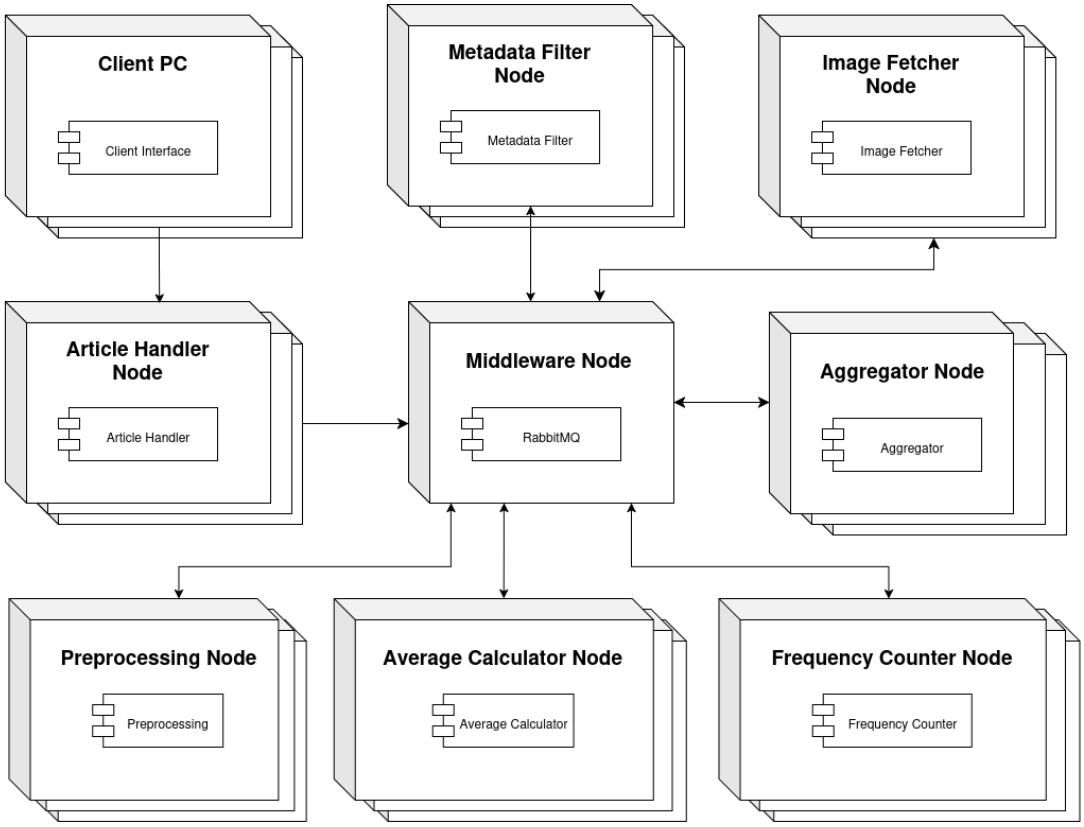



Figura 7: Despliegue de los nodos y servicios del sistema

El diagrama muestra los nodos principales: el cliente, los componentes de procesamiento y el middleware RabbitMQ. Las conexiones representan la comunicación entre ellos, y las siluetas superpuestas indican que algunos nodos pueden tener varias instancias.

### 6.2 Robustez

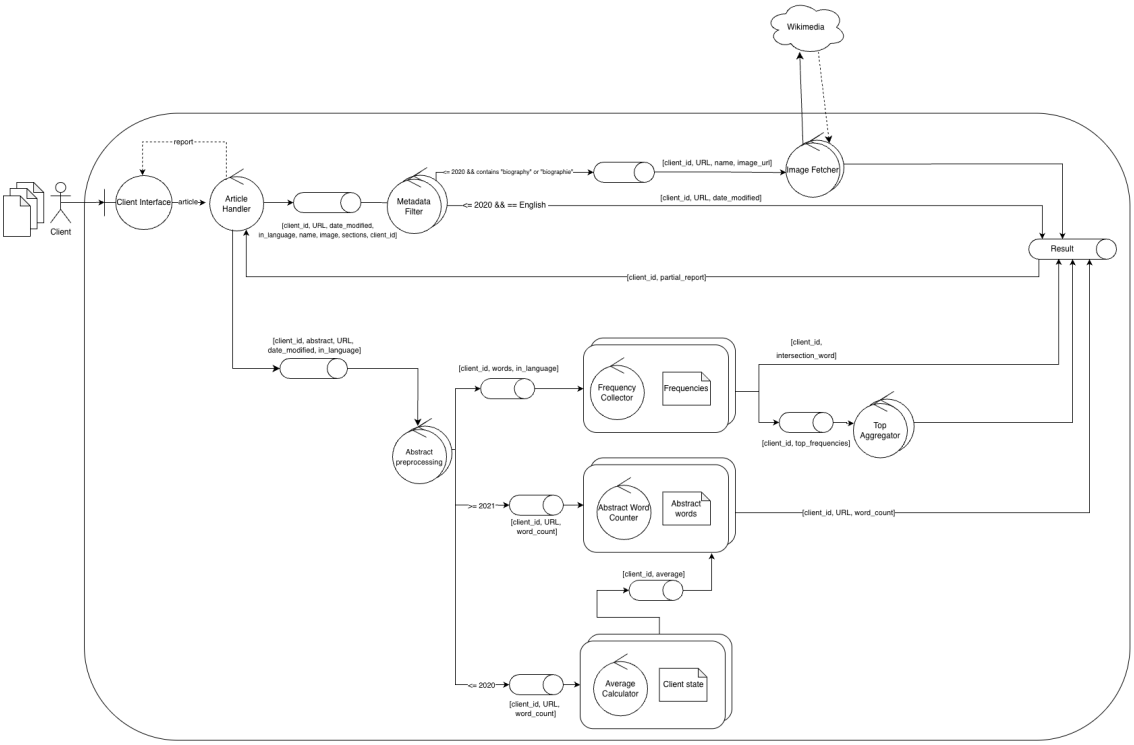



Figura 8: Flujo de datos, resultados y estado por cliente

Este diagrama muestra cómo los artículos se separan en flujos de metadatos y abstracts. Los metadatos se filtran para las consultas y, cuando corresponde, se solicita la imagen a Wikimedia. Los abstracts se procesan para calcular frecuencias y promedios. Los resultados se envían a la cola *Result* y se asocian con cada cliente mediante `client_id`.

Para responder al incremento en la carga de trabajo, el sistema permite el escalamiento horizontal:

- **Replicación de nodos:** Los componentes con mayor demanda computacional o de red (como `Image Fetcher`, `Frequency Counter` y `Average Calculator`) admiten múltiples instancias en paralelo.
- **Desacoplamiento mediante colas:** Las colas permiten repartir la carga entre trabajadores.

## 7 Escenarios

### 7.1 Procesamiento de una ejecución

Un cliente inicia una ejecución y envía artículos en inglés y francés junto con un identificador. *Article Handler* publica los mensajes de metadatos y abstracts. Los trabajadores procesan ambas ramas: las consultas 1, 2 y 3 pueden producir resultados parciales, mientras que las consultas 4 y 5 requieren completar los cálculos

globales correspondientes. Al terminar el envío, el cliente transmite `EOF`; la interfaz le entrega los resultados asociados a su identificador y cierra la ejecución cuando concluye el procesamiento.

### 7.2 Ejecuciones concurrentes

Varios clientes envían artículos al mismo tiempo. Los trabajadores consumen los mensajes de las colas y procesan las ejecuciones en paralelo. El `client_id` permite asociar cada resultado con el cliente correcto y evita mezclar los datos entre ejecuciones.

### 7.3 Terminación ordenada

Ante una señal `SIGTERM`, cada componente deja de aceptar trabajo nuevo, finaliza el procesamiento que ya inició y cierra sus conexiones de forma ordenada.

## 8 Tamaño y performance

### 8.1 Configuración de la prueba

La prueba de escalabilidad se configuró para procesar datos con tres clientes concurrentes. Los artículos se agrupan de a 100 registros. Se generan tres instancias de los nodos de metadatos y abstracts indicados, y dos instancias de *Image Fetcher*:

| **Parámetro** | **Valor** |
|---|---|
| Tamaño del lote (`batch-size`) | 100 |
| Clientes concurrentes | 3 |
| `metadata-filter` | 3 instancias |
| `abstract-preprocessing` | 3 instancias |
| `frequency-collector` | 3 instancias |
| `top-aggregator` | 3 instancias |
| `average-calculator` | 3 instancias |
| `abstract-word-counter` | 3 instancias |
| `image-fetcher` | 2 instancias |

### 8.2 Resultados observados

El tiempo total, medido desde el inicio del sistema hasta que los tres clientes recibieron todos sus resultados, fue de **16 min 32 s**. Los últimos eventos ocurridos durante la ejecución se resumen en la siguiente tabla:

| Tiempo | Cliente | Evento registrado |
|---|---|---|
| 10 min 38 s | `1-3` | Se recibió el `EOF` de la consulta 5; el registro indica progreso `4/5`. |
| 12 min 50 s | `1-2` | Se envió el archivo `185/185`; el cliente recibió todos sus resultados. También se registró el `EOF` de la consulta 2 (`5/5`). |
| 14 min 51 s | `1-1` | Se envió el archivo `185/185`; el cliente recibió todos sus resultados. También se registró el `EOF` de la consulta 2 (`5/5`). |
| 16 min 32 s | `1-3` | Se envió el archivo `185/185`; el tercer cliente recibió todos sus resultados y finalizó la ejecución global. Se registró el `EOF` de la consulta 2 (`5/5`). |

Tabla 2: Eventos y tiempos observados en la ejecución con tres clientes

# 9 Listado de tareas

La implementación se divide por consultas y componentes. Cada integrante implementará el manejo de `EOF` y el cierre ordenado ante `SIGTERM` para los componentes que tenga asignados.

## 9.1 Tareas por integrante

- **María Florencia Paisan — Servidor y consultas 1 y 2**
- **Ignacio Politti — Consultas 3 y 4**
- **Tomás Pérez D'Angelo — Cliente y consulta 5**

# A Apéndice

## A.1 Siglas y abreviaturas

- **API:** interfaz de programación de aplicaciones (*Application Programming Interface*).
- **DAG:** grafo acíclico dirigido (*Directed Acyclic Graph*).
- **EOF:** marcador de fin de entrada (*End of File*).
- **JSON:** formato de intercambio de datos (*JavaScript Object Notation*).
- **SIGTERM:** señal del sistema operativo para solicitar la terminación de un proceso.
- **URL:** localizador uniforme de recursos (*Uniform Resource Locator*).
- **en / fr:** códigos de idioma para inglés y francés respectivamente.

## A.2 Definiciones

- **Abstract:** resumen textual asociado a un artículo de Wikipedia.
- **Cliente:** proceso que envía una colección de artículos y recibe los resultados de una ejecución.
- `client_id`**:** identificador usado para asociar mensajes y resultados con la ejecución de un cliente.
- `EOF`**:** mensaje que indica que el cliente terminó de enviar artículos para una ejecución.
- **Middleware:** capa que abstrae la comunicación entre componentes mediante el intercambio de mensajes.
- **Stopword:** palabra trivial que se excluye de determinados análisis de frecuencia.
- **Trabajador:** instancia de un componente que consume mensajes de una cola y ejecuta una etapa de procesamiento.

## A.3 Principios de diseño

- **Separación de responsabilidades:** cada componente realiza una tarea delimitada, como filtrar metadatos, procesar abstracts o agregar resultados.
- **Comunicación desacoplada:** los componentes intercambian mensajes a través del middleware en lugar de depender de llamadas directas entre todos ellos.
- **Aislamiento por cliente:** los mensajes y resultados conservan el `client_id` para evitar mezclar ejecuciones.
- **Escalamiento horizontal:** aumentar la capacidad mediante réplicas de los trabajadores cuando crece la carga.
- **Terminación controlada:** propagar el fin de la entrada mediante `EOF` y atender `SIGTERM` con un cierre ordenado.